Introduction to Cardano Smart Contracts

Overview of Cardano Blockchain History

Charles Hoskinson, along with Jeremy Wood, co-founded Cardano, both were part of Ethereum before. In 2015, th

Ouroboros

Ouroboros relies on a **proof of stake** consensus. Rather than requiring nodes to engage in computationally intensi

Cardano Architecture

The blockchain model comprises several components. Users interact with the current ledger state by creating transa

Cardano offers several advantages over other chains:

Determinism: This feature enables transaction chaining.

Predictable Fees: There is no risk of pending transactions due to fee increases.

Sustainability: Unlike PoW, which is power-intensive, Cardano's approach is more sustainable.

Native Tokens: Tokens are stored on the ledger, allowing smart contracts to interact with them. Users have control ov

Scaling is currently the most significant challenge for the ecosystem. The ability to handle high volumes of users wi

Determinism and predictable fees to make a better user experience Why ***Determinism*** is important in smart cont

Alice, Bob and Raul are in a Bar, each has 100 ADA

Alice sends to Raul 50 ADA but the blockchain right now is super clogged

However Raul is already able to build a transaction to send 120 ADA to Bob because even if the blockchain is clogged,

Now even Bob can send 220 ADA to someone else, even before the transactions are confirmed, due to Cardano determin

Everything seems amazing, perfect. Where is the issue? What happens if for some reason Alice had her transaction

Why do predictable fees matter and what's their role in *determinism*?

On Bitcoin when you set inputs (what you spend), outputs (who gets the money) and fees you can get the transact

On Cardano there is no fee market, in this way, once you pay enough to cover the processing costs, that transaction

Even if it sounds cool, this leads to a problem, if there is no way of speeding up my transaction or making it possib

Importance and Applications of Smart Contracts

Smart contracts are a concept born alongside Ethereum, enabling the execution of code and interactions without a

However, history teaches us that some protocols have included backdoors within their smart contracts, leading to fu

Let's start with the basics.

What is a Smart Contract?

A smart contract is a decentralized software accessible to users on the blockchain, typically through a website interf

The essential components of a smart contract are:

Parties: Who can interact with the contract? Is it open to everyone, specific users, or owners of particular assets?
Actions: What operations can users perform with the contract? These could include depositing funds, creating NFTs,
Rules: Define the actions each party can take under specific conditions.
Data Fields: What data is involved in interactions with the contract, and how can each step of the interaction be trac

Applications

In a typical decentralized exchange (DEX) application, the parties are liquidity providers and traders. Liquidity pro

In a marketplace scenario, the parties are sellers and buyers. Sellers can sell assets, while buyers can buy them. Ru

On Cardano, specific actions might include ***Cancel Listing*** and ***Buy***. ***Selling/List*** is more of a smart contract i

A smart contract action involves a transaction where the smart contract is invoked in the inputs. If the smart contr

There can be many more smart contract applications, imagination is the limit, and some applications may work be

Smart Contract audits

Once a smart contract is developed and ready to launch an audit should be done, this is to ensure that the code is

But audits have a cost and sometimes early projects may not afford that much.

Opensource can be a way in order to launch a project asking for external reviews coming from the community, usua

Smart Contract risks

On Cardano there are some risks regarding smart contracts that we'll study better in the following chapters, but he

Double satisfaction attack: User may spend two inputs that require similar conditions
Dust attack: Users may add spam tokens in a smart contract making it impossible to retrieve funds from it
Spam Contract: A second smart contract can run together with the attacked one, making it possible to unlock funds fro
Datum attack: A datum of the contract may be corrupted making unspendable the funds inside the contract
backdoor: All the funds of the contract may be retrieved by someone who coded a backdoor

The cost of deploying a contract

Deploying a contract onto the blockchain carries no direct cost, allowing widespread accessibility. However, it's esse

Two years ago, if you asked me about the advantages of writing smart contracts on Cardano, I would have struggle

Composability: The ability to create a transaction involving multiple contracts and perform actions with each of them
User-Friendly: No longer requiring Haskell, languages like Aiken, Opshin, and more offer a user-friendly experience.
Liquid Staking: Thanks to Cardano staking, smart contracts can delegate ADA or keep funds staked with liquidity pr
UTxO Skills: While much of the focus has been on Ethereum Virtual Machine (EVM) smart contracts, the UTXO mo

If you're still interested in becoming a Cardano smart contract wizard after this introduction, we can continue in th

Installing and Configuring Cardano Development Tools Installing and Configuring Cardano Development Tools The
A hot Wallet: We are going to use a wallet to test our contracts, this wallet will be used to receive tADA. We'll never
A indexer account: Indexers are the ones that will provide us the APIs in order to interact with the chain, we won't
Lucid library: Lucid is not maintained anymore as a function and has been replaced by COMING SOON, however for
tADA: How are we going to test without having testnet ADA? let's not mess up real ADA
IDE: Personally I use Visual Studio Code as IDE, but any other editor is ok since we are going to
Cardano Node: This is NOT mandatory at all, as homework, we could try to set up a Cardano node and interact wit
*A new library is being built and once live it's going to be added in this book.
Hot wallets on Cardano When it comes to the wallet choice on Cardano the question we should ask ourselves is: De

[h!] Current Cardano wallets available, updated in Q2 24

Wallets	Desktop	Mobile	Website
Nami	X		https://www.namiwallet.io/
Eternl	X	X	https://eternl.io/
Begin		X	https://begin.is/
Vespr		X	https://vespr.xyz/
Lace	X		https://www.lace.io/
NuFi	X		https://nu.fi/
Yoroi	X	X	https://yoroi-wallet.com/
Flint	X	X	https://flint-wallet.com/
Gero	X		https://www.gerowallet.io/
Typhon	X		https://typhonwallet.io/

Test the wallet you like most and pick the one that gives you more user-friendly vibes for your use, a developer may
Setting Up and Connecting to Cardano Testnet So let's install a wallet and config for testnet
In this example, we'll install the Nami wallet that we can find https://www.namiwallet.io/here
Once we install the wallet we'll get 24 seed phrase words
Never share the seedphrase or store it on a cloud, use paper or different ways to store it, software can keep track of
$wallet_preview$
Let's set Nami to **testnet preview** and we'll finally get our wallet in testnet
Preview and Preprod testnets
Mainnet: This is the live network where real transactions occur using actual ADA. It's the primary arena where users
Preprod: Acting as a staging ground for major upgrades and releases, Preprod is a testing environment where develop
Preview: Serving as a testing environment to showcase upcoming features and functionality, Preview allows developers
Get tADA
In order to receive tADA we can use the official faucet from Cardano at the following https://docs.cardano.org/car
The process doesn't involve any payment and at the end of your testing, ideally, you should return tADA back so o
CIP30
To connect our wallet with any webpage we'll use CIP30 reference, we can find the list of methods to connect and i
[scale=0.3]CIP30
The steps to interact with a wallet following CIP30 are:
cardano.walletName.enable(): we get an API object as Promise, this will create a popup message to allow the walle
api.getBalance(): using the API object we got before, we get the total amount of Lovelace in the wallet (1 ADA = 10
api.signTx: Signing a tx that was built with Lucid or any other library we sign and interact with the blockchain
EXERCISE 1: Create a webpage with 2 buttons, 1 to enable the wallet connection and, a second button to view th
Interacting with Cardano node and Wallet APIs
CIP30 is not enough, what if we want to get the information regarding a specific NFT in our wallet? How to get th
We need an indexer. We could set one on our own or use a service, in this book we'll use **Maestro** as a service pro
Create a Maestro account
Head over https://dashboard.gomaestro.org/loginMaestro login page and create an account, here we'll be able to ge
[scale=0.3]maestro.png
Maestro is going to be our key to getting all the possible APIs in order to interact with Cardano, here are the possi
Get the history of an address with this ttps://docs.gomaestro.org/Indexer-API/Addresses/txs-by-addressAPI
Get all the assets of a specific policy
Get the address holding a specific ada handle
Get the history of holders for a specific NFT
and much more
Now that we have a way to interact with a wallet and APIs to query the Cardano blockchain we are ready to put o
Indexer alternatives
If you would like to explore additional API providers you should consider the following:
Blockfrost: The very first API provider for Cardano https://blockfrost.io/dashboardlink
Kupo: A tool that requires a node in order to host your own11 indexer https://github.com/CardanoSolutions/kupoGith
Db-sync: Additional indexer that requires a Cardano Node, this is the very first one that was created https://github.co
Now Let's code smart contracts.
EXERCISE 2: Head over to http://cnftlab.party/ and connect your testnet wallet, mint a collection of NFTs and t
Setting Up a Cardano node on Contabo Cloud VPS
Step 1: Provisioning a Contabo Cloud VPS
Sign up for a Contabo Cloud VPS plan, such as the "Cloud VPS L".
Once you have access to your VPS, log in via SSH.
Step 2: Downloading and Extracting Cardano node Software
Navigate to the official Cardano GitHub release page: https://github.com/input-output-hk/cardano-node/releases/tag/8
Download the Cardano Node software for macOS by running the following command:

```
wget https://github.com/input-output-hk/cardano-node/releases/download/8.1.2/cardano-node-8.1.2-
```

After the download completes, create a directory named "node" and extract the downloaded files into it:

```
mkdir node
tar xvzf cardano-node-8.1.2-macos.tar.gz -C node
```

Step 3: Setting Up Node Configuration
Create the necessary directories:

Understanding the eUTxO Model and Its Components Understanding the eUTxO Model and Its Components
Two popular record-keeping models in blockchain networks are the eUTXO (Extended Unspent Transaction Outpu
eUTXO Model In Cardano, each transaction spends outputs from prior transactions and generates new outputs for
Example 1. Alice gains 12.5 ADA through staking rewards, resulting in one eUTXO of 12.5 ADA.
2. Alice sends 1 ADA to Bob. Alice's wallet uses her eUTXO of 12.5 ADA, sending 1 ADA to Bob and receiving 11
3. If Bob had an eUTXO of 2 ADA before step 2, his wallet now shows a balance of 3 ADA from two eUTXOs.
Account/Balance Model The Account/Balance Model maintains the balance of each account as a global state. It ch
Example 1. Alice gains 5 ETH through mining, recorded in the system.
2. Alice sends 1 ETH to Bob, reducing her balance to 4 ETH.
3. Bob's balance increases by 1 ETH, so if he had 2 ETH initially, he now has 3 ETH.
Analogies
eUTXO Model: Similar to using paper bills, where each bill (eUTXO) can be spent once, and change is returned as n
Account/Balance Model: Similar to a bank's ATM/debit card system, where the bank ensures sufficient balance bef
Benefits
eUTXO Model
Scalability: Enables parallel transactions and scalability innovations.
Privacy: Provides higher privacy, especially with new addresses for each transaction.
Account/Balance Model
Simplicity: Easier for developers of complex smart contracts requiring state information.
Efficiency: More efficient as each transaction only validates account balance.
*Drawbacks **Account/Balance Model**: Susceptible to double-spending attacks, counteracted by an incrementing
Both models have trade-offs and are chosen based on specific blockchain needs. Some blockchains, like Hyperledger
Writing Transactions and Validating Inputs and Outputs
In this section, we'll analyze the components of a Cardano transaction and how it is built using a Cardano node an
Let's start with the following transaction:
https://preview.cexplorer.io/tx/a2fcdf32987ebb729ab8f63b377e360ecebcf4805713ff559d2b69bb3c543a01a2fcdf32987e
[scale=0.3]transactionTestnet
Let's analyze what we can see here:
On the **left**, we have the inputs of the transaction. We can see that we are spending 3 inputs containing tADA and
`5df4a4fb78650b5d8b0e761e0ade2c2ab2289b4feb97f6780b82d78b3d02bf70`
`f0e29aed793164ce8192aec7ec647b7e7747206d701b0dfd4a810414f7acb96b`
`5df4a4fb78650b5d8b0e761e0ade2c2ab2289b4feb97f6780b82d78b3d02bf70`
This means that for each transaction, we can obtain the transactions that generated the funds being spent directly
But that's not all; for each input, we are able to see from which address they come from, in this case `addr_test1qq`
On the **right** side, we can see the outputs of the transaction. Therefore, where is the tADA going?
We have 3 outputs:
The first output is sending 10,000 tADA and 1M tFLDT to `addr_test1qpku...sjuk35q`.
The second output is a change to `addr_test1qqwywxe3ag9sf3jjhk8hd...fhn5gh` sending back all the tADA that was l
The third output is a change to `addr_test1qqwywxe3ag9sf3jjhk8hd...fhn5gh` sending back all the NFTs and tokens t
Finally, we can see that this transaction was validated during epoch 623, block 2,270,567, and slot 53,865,862. It wa
Build a transaction with Cardano node
This part is not mandatory since it requires to run a Cardano node, however it is interesting to see how a transacti
Let's create a folder:
mkdir exercise01
Now we create the wallet with:
cardano-cli address key-gen –verification-key-file payment.vkey –signing-key-file payment.skey
To see the address, use the following command:
cardano-cli address build –payment-verification-key-file payment.vkey –out-file payment.addr
And then:
cat payment.addr
And we now see our address!
*Checking funds in wallet
Let's run the command:
cardano-cli query utxo –address ($catpayment.addr$) $--mainnet$
We will see that there are no transactions, so 0 ADA.
*Sending funds
Let's send some ADA to the wallet from our main wallet (try with 5 ADA) and run the command again to check th
cardano-cli query utxo –address ($catpayment.addr$) $--mainnet$
Now we should see some ADA. For instance, 5,000,000 lovelace means 5 ADA.
*Check funds of any address
If you want to check the ADA inside any wallet, the command becomes:
cardano-cli query utxo –address ADDRESSTOCHECKHERE –mainnet
The result is all the transactions containing ADA and NFTs in the address.
*Creating the raw transaction
Let's copy the transaction hash that contains the 5 ADA and the index:
cardano-cli transaction build-raw –fee 0 –tx-in HASHOFUNSPENTTRANSACTIONINDEX –tx-out ADDRESSRE
*Calculate the fee
We must calculate the fee according to the network parameters that we get with the following:
cardano-cli query protocol-parameters –mainnet –out-file protocol.json
Now:
cardano-cli transaction calculate-min-fee –tx-body-file matx.raw –tx-in-count 1 –tx-out-count 2 –witness-count 1 –n
And we get the fee we should pay at least.
*Build the final transaction
Now we can finally build the complete transaction with:
cardano-cli transaction build-raw –fee $\mathrm{FEE}_W E_C ALCULATED --tx - inHASHOFUNSPENTTRANSACTIO$
At this point, the transaction is finished. We must sign it with the key to prove we are the owners.
*Signing the transaction and submitting it to the blockchain

Cardano Smart Contract Languages Introduction to Plutus Language
Plutus is the native smart contract language for Cardano. It is a Turing-complete language written in Haskell, and
The Plutus Playground has been recently updated, providing a platform for developing smart contract applications
Plutus Tx Compiler The key technology that facilitates this is Plutus Tx, which acts as a compiler from Haskell to
Handling Haskell's Complexity Haskell, known for its complexity and numerous sophisticated extensions, is simplifi
"Fortunately, the design of GHC, the primary Haskell compiler, makes this possible. GHC has a very simple repres
While Haskell's type system is complex, Plutus Tx leverages a subset of it, sufficient for blockchain needs. However
Compilation Pipeline The compilation process involves multiple stages, utilizing intermediate languages to break do
GHC: Haskell to GHC Core
Plutus Tx compiler: GHC Core to Plutus IR
Plutus IR compiler: Plutus IR to Typed Plutus Core
Type eraser: Typed Plutus Core to Untyped Plutus Core
Plutus IR, closely aligned with GHC Core, reduces the logic within the plug-in and allows for independent testing o
Integration with GHC Plutus Tx integrates into the GHC compilation pipeline through GHC plug-ins, which modif
"Because we are able to modify the program GHC is compiling, we have an obvious place to put the output of the
Runtime Considerations Dynamic elements of smart contracts, such as participant details or transaction amounts, r
References For more detailed information, visit the Plutus Playground and follow the development on https://githu
Setting Up the Environment
To build the Plutus development environment on an Ubuntu 20.04 VM, follow these steps:
Installing Haskell and Components

Create and start a VM:

Create a VM with at least 8 GB of RAM and 100 GB of storage, and select Ubuntu 64-bit.
Install Ubuntu 20.04 on the VM.

Update and upgrade the system:

```
sudo apt update
sudo apt upgrade -y
```

Install Haskell:

```
sudo apt-get install haskell-platform
```

Install `ghcup` and required dependencies:

```
sudo apt install curl
sudo apt install build-essential curl libffi-dev libffi6 libgmp-dev
libgmp10 libncurses-dev libncurses5 libtinfo5 libsodium-dev
curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

Restart your terminal.
Check the Haskell version:

```
ghc --version
cabal --version
```

Clone the Plutus repositories:

```
sudo apt install git
mkdir cardano
cd cardano
git clone https://github.com/input-output-hk/plutus
git clone https://github.com/input-output-hk/plutus-pioneer-program
```

Run `cabal update` and `cabal build` each time you start on a different section of the homework.
Installing `nix-shell` and Plutus Binaries

Install the cache:

```
sudo mkdir /etc/nix
```

Create a file named `nix.conf` in `/etc/nix/` with the following content:

```
substituters        = https://hydra.iohk.io https://iohk.cachix.org
trusted-public-keys = hydra.iohk.io:f/Ea+s+dFdN+3Y/
G+FDgSq+a5NEWhJGzdjvKNGv0/EQ=iohk.cachix.org-1:
DpRUyj7h7V830dp/i6Nti+NEO2/nhblbov/8MW7Rqoo=
 cache.nixos.org-1:6NCHdD59X431o0gWypbMrAURkbJ16ZPMQFGspcDShjY=
```

Install `nix`:

```
curl -L https://nixos.org/nix/install | sh
```

After installation, set up the environment variables as instructed. Typically, this involves running:

```
. /home/(user)/.nix-profile/etc/profile.d/nix.sh
```

You may need to add `/.nix-profile/bin` to your `/etc/environment` file and restart the computer.

Build Plutus with `nix-shell`:

```
cd ~/cardano/plutus
git checkout 3746610e53654a1167aeb4c6294c6096d16b0502
nix build -f default.nix
plutus.haskell.packages.plutus-core.components.library
```

This process may take a while and may produce a warning about dumping a very large path.
The first run of `nix-shell` will also take some time. The first video in the course will guide you through starting the P
Writing a Simple Smart Contract in Plutus Tx
Here is a simple example of a smart contract in Plutus Tx. This contract verifies that the number passed in the red

Smart Contract Runtime and Execution Smart Contract Runtime and Execution

This part has been updated to be Chang compatible. The code here is already compatible with DReps, the new Aiken

Overview of Cardano's Smart Contract Execution Model

Smart contracts on Cardano are simple constructs based on **validator scripts**. These scripts define the logic or ru

Validator scripts can access the **transaction context** (e.g., who signed it, and which assets are sent to/from where

Validator scripts are executed with three main arguments:

Datum: Attached to the output locked by the script, often carrying state.

Redeemer: Attached to the spending input, typically providing an input to the script. For example, a validator can ap

Context: Contains transaction-level information, used to assert properties like "Bob signed it."

Transaction context properties:

Understanding the Transaction Context Table

Inputs: These represent UTXOs being *spent* in the transaction. In the UTXO model, every transaction produces o

Outputs: These are the new UTXOs created by the transaction, ready to be used as inputs in subsequent transact

Fees: The amount of lovelace spent for transaction execution. This value is predictable and depends on the transac

Minted Value: This indicates any minting or burning of tokens that occurs in the transaction.

Certificates: Information about stake operations. For example:

Registering a stake key.

Delegating to a DRep.

Withdrawals: Rewards withdrawn from stake keys. For example, if you use a wallet like Lace, which allows deleg

Valid Range: A time frame during which the transaction is valid. This is useful for ensuring a transaction only ex

Signatories: A list of hashes representing who signed the transaction. This is essential for multi-signature transac

Redeemers: A list of redeemers used by the contracts executed in the transaction.

ID: The transaction hash, uniquely identifying the transaction.

During runtime, each validator checks its *datum*, *redeemer*, and the *context* (or local transaction state).

Important exercise: Imagine a transaction that:

Spends **three different UTXOs** from the same contract.

Withdraws staking rewards from a stake key contract.

Mints **two different tokens** under separate contract policies.

Questions:

How many unique contracts are executed?

How many datums are present?

How many redeemers?

Answer: After a blank page—*Take your time; don't rush!*

Real answer:
4
3
6

Did you get them? Let's try to understand why. Spending inputs from a contract, even if it's coming from the sam
Withdraw and minting contract do not have datums, they have redeemers. You can mint or withdraw from the san
It will be easier to understand at the end of this chapter, you need to take home this:
SPEND contracts are executed once for each input, while minting and withdrawals contracts are executed once for
Transaction Verification and Script Validation
To understand transaction validation on Cardano, it is crucial to recognize that even if a script validates a transact
Two-Phase Validation Mechanism
Cardano employs a two-phase validation scheme to minimize uncompensated work for nodes, ensuring efficiency an

Phase 1: Structural Validation
Checks if the transaction is correctly constructed.
Ensures that the transaction can pay its processing fee.
If Phase 1 fails, the transaction is immediately discarded without running any scripts.
Phase 2: Script Execution
Executes the scripts included in the transaction.
If a script fails, the transaction fails, and collateral is used to compensate nodes.

The Role of Collateral collateral.png
Collateral ensures that nodes are compensated for their work if Phase 2 validation fails. It acts as a monetary guar

Collateral Inputs: The collateral amount is determined by the total balance of UTXOs marked as collateral inputs.
Safety for Honest Users: Collateral is not collected if a transaction succeeds or is invalid at Phase 1.
Deterministic Costs: Cardano's deterministic design allows users to calculate execution costs and collateral requirem
Vasil Upgrade Improvement: Developers can specify a change address for script collateral, ensuring only the require

Importance of Collateral
Without collateral, malicious actors could exploit the network by flooding it with invalid transactions at little cost.

Transactions calling non-native (Phase 2) smart contracts include sufficient collateral to cover potential failure costs.
Denial of Service (DoS) attacks become prohibitively expensive.
Honest users never lose their collateral as long as transactions are valid and successful.

Technical Details
Phase 2 scripts on Cardano can perform arbitrary computations and require a budget of execution units (ExUnits)

Multi-Signature Scripts: Introduced in the Shelley era, Phase 1 scripts follow deterministic ledger rules, enabling str
ExUnit Budgeting: Phase 2 scripts require a resource budget for metrics like memory usage and execution steps, ensu

The Cardano testnet provides a safe environment with free test ADA, enabling developers to rigorously test smart
Debugging and Troubleshooting Smart Contracts
Debugging smart contracts can be challenging, but interacting directly with the blockchain is not always necessary.
Debugging with Aiken
Aiken offers first-class support for unit tests and property-based tests, allowing developers to write and execute test
Unit Tests
Unit tests in Aiken are written using the 'test' keyword. A test is a named function that takes no arguments and re

```
test foo() {
  1 + 1 == 2
}
```

Unit tests can call functions and use constants, and they execute on the same virtual machine used for on-chain co
Example Unit Tests

```
lib/example.ak
fn add_one(n: Int) -> Int {
  n + 1
}

test add_one_1() {
  add_one(0) == 1
}

test add_one_2() {
  add_one(-42) == -41
}
```

Running 'aiken check' generates a report grouping tests by module and displaying the memory and CPU execution
Tests in Aiken can be as complex as necessary, without the execution limits imposed on on-chain scripts.
Trace and Debugging
Aiken supports debugging via CBOR diagnostic traces. Developers can use 'trace' to print diagnostic data (e.g., int

```
// An Int becomes a CBOR int
trace cbor.diagnostic(42)

// A ByteArray becomes a CBOR bytestring
trace cbor.diagnostic("foo")
```

This feature provides valuable insights during debugging by simulating breakpoints.
Advanced Debugging with Gastronomy [scale=0.3]gastronomy.png Gastronomy, developed by Sundae Labs, is a pow
Features
Stores the state of the machine at every execution step.
Allows stepping forward and backward to analyze script behavior.
Provides a user-friendly interface for debugging.
Quick Start
CLI Tool:

```
gastronomy-cli run test_data/fibonacci.uplc 03
N - Advance to the next step
```

Getting Started with Cardano Smart Contract Development Key Concepts and Terminology
In this chapter, we will get hands-on experience by writing a smart contract in Aiken and the corresponding off-cha
At the end of the chapter, you will also find exercises to test your understanding and skills.
Key Concepts
Here is a list of key concepts and terminology that will help you:
Collateral: A specific UTxO required to interact with smart contracts. It ensures that nodes are compensated in case
Script Hash: A unique hash that identifies the smart contract, allowing transactions to reference it securely.
Locked UTxO: A transaction output that holds funds, NFTs, or tokens locked within a contract. These can only be s
CIP69 Contract: A contract where the datum is optional. It behaves like a user, allowing it to receive funds. Howeve
Multipurpose Script: A versatile script that can perform multiple functions, such as locking funds, minting NFTs, a
Writing Your First Smart Contract
Let's write and execute a smart contract on Cardano in 10 minutes. Yes, you read that right.
You can find code supporting this tutorial on Aiken's main repository.
Covered in this tutorial
Writing a basic Aiken validator;
Writing & running tests with Aiken;
Troubleshooting smart contracts.
When encountering an unfamiliar syntax or concept, do not hesitate to refer to the language-tour for details and ex
Pre-requisites We'll use Aiken to write the script, so make sure the command-line tool is installed already. Otherwi
Scaffolding First, let's create a new Aiken project:

```
aiken new aiken-lang/hello-world
cd hello-world
```

This command scaffolds an Aiken project. In particular, it creates a `lib` and `validators` folder in which you can p

```
./hello-world

 README.md
 aiken.toml
 lib
 validators
```

Using the standard library We'll use the standard library for writing our validator. Fortunately, `aiken new` did aut

```
aiken.toml
name = "aiken-lang/hello-world"
version = "0.0.0"
license  = "Apache-2.0"
description = "Aiken contracts for project 'aiken-lang/hello-world'"

[repository]
user = 'aiken-lang'
project = 'hello-world'
platform = 'github'

[[dependencies]]
name = "aiken-lang/stdlib"
version = "v2"
source = "github"
```

Now, running `aiken check`, we should see dependencies being downloaded. That shouldn't take long.

```
 aiken check
    Compiling aiken-lang/hello-world 1.0.0 (examples/hello-world/)
    Resolving aiken-lang/hello-world
      Fetched 1 package in 0.01s from cache
    Compiling aiken-lang/stdlib v2 (/Users/aiken/Documents/aiken-lang/hello-world/build/packages/aik
      Summary 0 errors, 0 warnings
```

Our First Validator Let's write our first validator as `validators/hello_world.ak`:

```
validators/hello_world.ak
use aiken/collection/list
use aiken/crypto.{VerificationKeyHash}
use cardano/transaction.{OutputReference, Transaction}

pub type Datum {
  owner: VerificationKeyHash,
}

pub type Redeemer {
  msg: ByteArray,
}

validator hello_world {
  spend(
    datum: Option<Datum>,
    redeemer: Redeemer,
    _own_ref: OutputReference,
    self: Transaction,
  ) {
    expect Some(Datum { owner }) = datum
    let must_say_hello = redeemer.msg == "Hello, World!"
    let must_be_signed = list.has(self.extra_signatories, owner)
    must_say_hello && must_be_signed
```

Building a Marketplace Defining Requirements for a Marketplace Contract

In this section, we define the essential requirements for our marketplace contract. The contract should include the f

Owner's Actions: An owner can cancel or create a listing in the contract.

Buyer Limitation: A buyer can purchase only one NFT at a time to avoid double-spending, a potential issue to be ex

NFT Transfer: Ensure that the NFT is correctly transferred to the buyer upon purchase.

Seller Payment: The contract must ensure that the seller receives the correct payment.

Marketplace Payment: The marketplace fee must be paid correctly.

Royalties: If royalties are present, they must be paid to the designated address.

The contract will store the following information for each NFT listing:

`nSeller` :: `PubKeyHash` `the address to pay`
`nPrice` :: `Integer` `the amount`
`nCurrency` :: `CurrencySymbol` `the policy of the token`
`nToken` :: `TokenName` `the asset name of the token`
`nRoyalty` :: `PubKeyHash` `the address of the royalty recipient`
`nRoyaltyPercent` :: `Integer` `the percentage of royalty`

Implementing Marketplace Contracts

To implement marketplace contracts, we will begin by analyzing the contract implementation in **PlutusTX**. We ca

In the repository, we can see how the marketplace contract is implemented in `Onchain.hs`, which defines the main

PlutusTX Code for Marketplace Contract

We provide the following code implementation for the market contract in Plutus:

[language=haskell, caption=Plutus Code for Marketplace Contract] - LANGUAGE DataKinds - - LANGUAGE No
module Market.Onchain (apiBuyScript , buyScriptAsShortBs , typedBuyValidator , Sale , buyValidator , nftDatum
import qualified Data.ByteString.Lazy as LB import qualified Data.ByteString.Short as SBS import Codec.Serialise
import Cardano.Api.Shelley (PlutusScript (..), PlutusScriptV1) import qualified PlutusTx import PlutusTx.Prelud
import Market.Types (NFTSale(..), SaleAction(..))
- INLINABLE nftDatum - nftDatum :: TxOut -¿ (DatumHash -¿ Maybe Datum) -¿ Maybe NFTSale nftDatum o f
- INLINABLE ensureOnlyOneScriptInput - ensureOnlyOneScriptInput :: ScriptContext -¿ Bool ensureOnlyOneScri
- INLINABLE mkBuyValidator - mkBuyValidator :: PubKeyHash -¿ NFTSale -¿ SaleAction -¿ ScriptContext -¿ Bo
tn :: TokenName tn = nToken nfts
cs :: CurrencySymbol cs = nCurrency nfts
seller :: PubKeyHash seller = nSeller nfts
sig :: PubKeyHash sig = case txInfoSignatories info of [pubKeyHash] -¿ pubKeyHash _ > *error*()
price :: Integer price = nPrice nfts
checkNFTOut :: Bool checkNFTOut = valueOf (valuePaidTo info sig) cs tn == 1
marketplacePercent :: Integer marketplacePercent = 20
marketplaceFee :: Ratio Integer marketplaceFee = max ($1_0 00_0 00$
checkMarketplaceFee :: Bool checkMarketplaceFee = fromInteger (Ada.getLovelace (Ada.fromValue (valuePaidTo i
royaltyFee :: Ratio Integer royaltyFee = max ($1_0 00_0 00$
checkRoyaltyFee :: Bool checkRoyaltyFee = if nRoyaltyPercent nfts ¿ 0 then fromInteger (Ada.getLovelace (Ada.fro
checkSellerOut :: Bool checkSellerOut = fromInteger (Ada.getLovelace (Ada.fromValue (valuePaidTo info seller))) ¿
checkCloser :: Bool checkCloser = txSignedBy info seller
onlyOneScriptInput :: Bool onlyOneScriptInput = ensureOnlyOneScriptInput ctx
data Sale instance Scripts.ValidatorTypes Sale where type instance DatumType Sale = NFTSale type instance Red
typedBuyValidator :: PubKeyHash -¿ Scripts.TypedValidator Sale typedBuyValidator pkh = Scripts.mkTypedValic

(PlutusTx.compile [—— wrap ——]) where wrap = Scripts.wrapValidator @NFTSale @SaleAction
buyValidator :: PubKeyHash -¿ Validator buyValidator = Scripts.validatorScript . typedBuyValidator
buyScript :: PubKeyHash -¿ Plutus.Script buyScript = Ledger.unValidatorScript . buyValidator
buyScriptAsShortBs :: PubKeyHash -¿ SBS.ShortByteString buyScriptAsShortBs = SBS.toShort . LB.toStrict . se
apiBuyScript :: PubKeyHash -¿ PlutusScript PlutusScriptV1 apiBuyScript = PlutusScriptSerialised . buyScriptAs

Types.hs Code for Marketplace Contract

Here is the code for `types.hs`, which defines the datum structures used in the marketplace contract:

[language=haskell, caption=Types Code for Marketplace Contract] - LANGUAGE DeriveAnyClass - - LANGUAGE
module Market.Types (NFTSale (..) , SaleAction (..) , SaleSchema , StartParams (..) , BuyParams (..)) where
import Data.Aeson (ToJSON, FromJSON) import GHC.Generics (Generic) import Prelude (Show (..)) import qua
import Schema (ToSchema) import qualified PlutusTx import PlutusTx.Prelude as Plutus (Eq(..), (), Integer) im
– This is the datum type, carrying the previous validator params data NFTSale = NFTSale nSeller :: !PubKeyHas
instance Eq NFTSale where - INLINABLE (==) - a == b = (nSeller a == nSeller b) (nPrice a == nPrice b) (n
PlutusTx.makeIsDataIndexed ”NFTSale [('NFTSale, 0)] PlutusTx.makeLift ”NFTSale
data SaleAction = Buy — Close deriving Show
PlutusTx.makeIsDataIndexed ”SaleAction [('Buy, 0), ('Close, 1)] PlutusTx.makeLift ”SaleAction
– We define two different params for the two endpoints start and buy with the minimal info needed. – Therefore th
For BuyParams we ommit seller and price because we can read that in datum which can be obtained with just cs a
data BuyParams = BuyParams bCs :: CurrencySymbol , bTn :: TokenName deriving (Pr.Eq, Pr.Ord, Show, Gen
data StartParams = StartParams sPrice :: Integer , sCs :: CurrencySymbol , sTn :: TokenName , sRoyaltyAddress
type SaleSchema = Endpoint ”close” BuyParams . Endpoint ”buy” BuyParams . Endpoint ”start” StartParams T

A buyer can purchase the NFT by fulfilling the following conditions:

Paying the correct amount to the seller.
Paying the applicable fees to the marketplace.
Paying royalties, if any are set.
Purchasing only one NFT at a time.

You don't need to fully grasp every detail of the contract at this moment. This book is here to guide you through t

Marketplace in Helios

To implement the marketplace contract in Helios, we can follow a similar structure to the Plutus contract but using

Defining the Contract Name First, we define the name of the contract as follows:

[language=haskell, caption=Contract Name Definition] spending marketplace

Defining the Datum Next, we define the `Datum` which holds the buy data for each NFT sale listing. The datum will

Governance Smart Contracts

Introduction to On-Chain Governance

Cardano is advancing towards a decentralized governance model designed to empower community participation and

Cardano Ledger Eras

Cardano has undergone various ledger era upgrades – Byron, Shelley, Allegra, Mary, Alonzo, Babbage, and the curr

[ht]

Cardano

Era	Key Features	Consensus	Hard Fork Event
Byron	Proof of stake	Ouroboros Classic / PBFT	Genesis / Byron reboot
Shelley	Decentralized block production	TPraos	Shelley HF
Allegra	Token locking	TPraos	Allegra HF
Mary	Native tokens	TPraos	Mary HF
Alonzo	Plutus smart contracts	TPraos	Alonzo HF
Babbage	PlutusV2, Reference inputs, Inline datums	Praos	Vasil
Conway	Decentralized governance	Praos	Chang HF

Transitioning from one era to the next is triggered by an update proposal that updates the protocol version. The C

The Governance Bodies

Cardano's governance model is built on three governance bodies that collaborate through a formalized voting syste

Ada Holders Ada holders are at the core of Cardano's governance, wielding critical voting power and the ability to

Delegation of Voting Power: Ada holders can delegate their voting rights to DReps to aggregate their influence. Th

Participation as DReps: Ada holders can register as DReps, locking a deposit of `dRepDeposit`, allowing them to dir

Submitting Governance Actions: Any Ada holder can propose governance actions on the blockchain network. To in

Influence on Network Operations: By delegating to specific SPOs, Ada holders indirectly influence the operational

Each associated stake key can issue two types of delegation certificates: one for delegation to a stake pool and the o

Delegated Representatives (DReps) DReps serve as officials representing Ada holders in the governance system. Eve

Proposing and Debating: DReps propose and debate changes to the Cardano protocol. They gather feedback from t

Voting on Proposals: DReps vote on proposals based on the priorities and interests of the Ada holders they represen

Voting Power: DReps' voting power is proportional to the stake delegated to them.

Stake Pool Operators (SPOs) SPOs are fundamental to the network's operational integrity and also play a crucial r

Maintaining network infrastructure and ensuring stable operation.

Participating in governance discussions, particularly on technical aspects.

Voting on governance actions with power based on their active stake.

Implementing approved changes by upgrading their infrastructure.

Constitutional Committee (CC) The Constitutional Committee oversees and guides the governance process to adhe

Interprets the Cardano Constitution and ensures governance actions align with it.

Provides a system of checks and balances by reviewing the constitutionality of governance actions.

Ensures transparency and fairness in the governance process.

Each CC member has one vote.

The Decision-Making Process

The governance process involves several stages:

Off-Chain Discussion: The majority of discussions and consensus-building about specific governance issues happens o

Governance Action Submission: Any Ada holder can submit a governance action as long as a sufficient ada deposit

Voting: Decisions on governance actions are made through a democratic voting process involving DReps, SPOs, and th

Enactment: Governance actions are checked for ratification only at the epoch boundary. Once ratified, actions will be

Governance Transactions and Their Structure

Types of Governance Actions

In the Conway ledger era, any ada holder can submit a Governance action, which is the on-chain method to put a p

[ht]

Types of Governance A

Governance Action	Description
NoConfidence	A motion to create a state of no-confidence in the current constitutional committee
UpdateCommittee	Changes to the members of the constitutional committee and/or to its signature threshold and/or terms
NewConstitution	A modification to the off-chain Constitution and/or the proposal policy script
TriggerHF	Triggers a non-backwards compatible upgrade of the network; requires a prior software upgrade
ChangePParams	A change to one or more updatable protocol parameters, excluding changes to major or minor protocol versions
TreasuryWdrl	Movements from the treasury
Info	An action that has no effect on-chain, other than an on-chain record

Ratification Thresholds

Each type of governance action requires meeting a different mix of voting thresholds to be ratified:

[ht]

Ratification Thresholds for Governance

Governance Action Type	CC	DReps	SPOs
Motion of no-confidence	-	0.67	0.51
Update committee/threshold (normal state)	-	0.67	0.51
Update committee/threshold (state of no-confidence)	-	0.60	0.51
New Constitution or Guardrails Script	2/3	0.75	-
Hard-fork initiation	2/3	0.60	0.51
Protocol parameter changes, network group	2/3	0.67	-
Protocol parameter changes, economic group	2/3	0.67	-
Protocol parameter changes, technical group	2/3	0.67	-
Protocol parameter changes, governance group	2/3	0.75	-
Treasury withdrawal	2/3	0.67	-
Info	2/3	1	1

Some parameters (from different groups) are relevant to security properties of the system. Any proposal attempting

Governance Action Lifecycle

A proposed governance action has a lifespan of 6 epochs, as defined by the protocol parameter `govActionLifetime`

At every epoch boundary within the governance action lifetime, proposals and votes are snapshotted. The `DRepPul`

Conclusions

Recap of Key Concepts and Takeaways Cardano's UTXO architecture allows for both native scripts and smart cont

It is also important to note that, while Haskell was once considered the primary language for programming on Card

Resources for Further Learning and Exploration Here are some useful resources to continue your journey in Cardan

[ht]

Useful Resources for Cardano Smart C

Resource	Link
Aiken Page	https://aiken-lang.org/
Aiken Docs	https://aiken-lang.github.io/stdlib/
Aiken Discord	https://discord.com/invite/Vc3x8N9nz2
Aiken GitHub	https://github.com/aiken-lang/aiken
Helios Page	https://www.helios-lang.io/
Helios Discord	https://discord.com/invite/XTwPrvB25q
Helios GitHub	https://github.com/orgs/HeliosLang/repositories
Pluts Docs	https://pluts.harmoniclabs.tech/
Pluts GitHub	https://github.com/HarmonicLabs/plu-ts
Anastasia Labs GitHub	https://github.com/Anastasia-Labs
Anastasia Labs Discord	https://discord.gg/VDXbPkcmjT
Mesh Page	https://meshjs.dev/
Mesh Discord	https://discord.com/invite/WvnCNqmAxy
Opshin Page	https://opshin.dev/
Opshin Book	https://book.opshin.dev/

Contributing to the Cardano Smart Contract Ecosystem The best way to contribute to the Cardano smart contract

Dear builder, if you reach this part let me tell you that I admire your passion and dedication. Please make sure to

This book is only the starting block of your journey as Cardano Developer

EXERCISE solutions Solutions Solutions
Exercise 1 *You can find the solution file for easy copy and paste:* https://github.com/elRaulito/eUTxO-Fundament
HTML sensitive=true, keywords=, otherkeywords=¡, ¿, /, =, morecomment=[s]¡!—¿, morestring=[b]", morestring=
basicstyle=, commentstyle=gray, stringstyle=blue, keywordstyle=red, showstringspaces=false, tabsiz
[]sections/solutions/index.html
Exercise 2 *You can find the solution file for easy copy and paste:* https://github.com/elRaulito/eUTxO-Fundament
Python keywords=import, def, while, True, if, else, break, print, with, as, not, None, keywordstyle=blue**, ndkeywo**
basicstyle=, commentstyle=gray, stringstyle=green, keywordstyle=blue, showstringspaces=false, tabs
[]sections/solutions/get$_a ddresses.py$
Exercise 4 *You can find the solution file for easy copy and paste:* https://github.com/elRaulito/eUTxO-Fundament
basicstyle=, commentstyle=gray, stringstyle=red, keywordstyle=blue, showstringspaces=false, tabsi
[]sections/solutions/nft.ak
Exercise 5 ***You can find the solution file for easy copy and paste:*** **https://github.com/elRaulito/eUT**
basicstyle=, commentstyle=gray, stringstyle=red, keywordstyle=blue, showstringspaces=false, tabsi
[]sections/solutions/cip69.ak
Exercise 6 ***You can find the solution file for easy copy and paste:*** **https://github.com/elRaulito/eUT**
basicstyle=, commentstyle=gray, stringstyle=red, keywordstyle=blue, showstringspaces=false, tabsi
[]sections/solutions/fraction.ak
Exercise 7 ***You can find the solution file for easy copy and paste:*** **https://github.com/elRaulito/cnft**
HTML sensitive=true, keywords=, otherkeywords=¡, ¿, /, =, morecomment=[s]¡!—¿, morestring=[b
basicstyle=, commentstyle=gray, stringstyle=blue, keywordstyle=red, showstringspaces=false, tabsi
[]sections/solutions/delist.js
Determinism name=Determinism, description=Transaction and blockchain behavior are predictable
CIP name=CIP, description=Cardano Improvement Proposals that if approved can change the curr
inputs name=inputs, description=Inputs in a UTXO model transaction specify which unspent outpu
epoch name=epoch, description=An epoch in Cardano is a fixed period during which a set of blocks
block name=block, description=A block in Cardano is a record of transactions and other informatio
slot name=slot, description=A slot is a smaller time unit within an epoch. An epoch is divided into